

Proyecto 02: YAPar

Generador de Analizadores Sintácticos SLR

Universidad del Valle de Guatemala

Facultad de Ingeniería: CC3071 Diseño de Lenguajes de Programación

Semestre I, 2026

Nombre	Carné	Responsabilidad principal
Diego Lopez	23747	Lector .yalp, modelos de gramática, tests
Jennifer Toxcon	21276	Generador SLR, FIRST/FOLLOW, autómatas LR(0)
Roberto Barreda	23354	CLI, runtime del parser, generación de código, GUI

Repositorio: <https://github.com/bar23354/PRY2-DLP>

1. Descripción del proyecto

YAPar es un generador de analizadores sintácticos SLR implementado en Python. Toma como entrada una gramática libre de contexto en lenguaje YAPar (.yalp) y una especificación léxica YALex (.yal), valida la consistencia de tokens entre ambos archivos y produce:

- Un archivo **theParser.py** completamente autónomo con el analizador sintáctico.
- El autómata LR(0) visualizado como imagen PNG o PDF.
- Detección y reporte de conflictos SLR, errores léxicos y sintácticos con posición.

2. Requisitos

- Python 3.10 o superior.
- Dependencias: `pip install -r requirements.txt` (pytest, customtkinter, Pillow, graphviz, matplotlib, networkx).
- Graphviz opcional para mejor resolución del diagrama LR(0): `winget install Graphviz.Graphviz`.

3. Diseño del software

3.1 Arquitectura por capas

El proyecto sigue una arquitectura en capas separadas por responsabilidad. Cada módulo expone una interfaz pública mínima y no conoce los detalles de implementación de los demás.

Módulo	Responsabilidad
src/grammarModel.py	Define los tipos inmutables Production y Grammar usados como contrato entre todas las capas.

Módulo	Responsabilidad
src/yalpReader.py	Elimina comentarios /* */, separa secciones con %%, procesa %token e IGNORE, parsea producciones y valida nombres y símbolos. Sin expresiones regulares.
src/yalexReader.py	Lee un archivo .yal y extrae los tokens definidos después de '->'. Valida que los tokens del .yalp existan también en el lexer.
src/slGenerator.py	Implementa el algoritmo SLR completo: aumentación, FIRST, FOLLOW, clausura, goto, estados LR(0), tablas ACTION/GOTO y detección de conflictos.
src/lr0Visualizer.py	Genera el diagrama del autómata LR(0) con Graphviz si está disponible; de lo contrario usa matplotlib + networkx como respaldo.
src/parserRuntime.py	Ciclo principal del parser: shift, reduce, accept y reporte de errores léxicos/sintácticos con número de línea y columna.
src/codeGenerator.py	Serializa las tablas ACTION/GOTO como literales Python y las inserta en una plantilla para producir theParser.py autocontenido.
src/lexerAdapter.py	Tokenizador simple: mapea palabras del archivo de entrada a tipos de token registrando posición (línea, columna).
src/guiApp.py	Interfaz gráfica con siete pestañas: Gramática, FIRST/FOLLOW, Estados LR(0), Tablas, Autómata, Generar Parser y Probar Parser.
yapar.py	Punto de entrada CLI: valida argumentos, orquesta el pipeline completo y reporta errores al usuario.

3.2 Flujo de datos

- **1. Lectura:** parseYalp() lee el .yalp y devuelve un objeto Grammar con tokens, tokens ignorados, producciones y símbolo inicial.
- **2. Validación léxica:** validateTokensInYalex() verifica que cada token del .yalp esté definido en el .yal.
- **3. Generación SLR:** buildSlrParserTable() aumenta la gramática, calcula FIRST/FOLLOW, construye los estados LR(0) y llena ACTION/GOTO. Si hay conflictos lanza SLRConflictError.
- **4. Visualización:** generateLr0Diagram() produce el PNG del autómata.
- **5. Generación de código:** generateParserFile() escribe theParser.py con las tablas serializadas.
- **6. Ejecución:** theParser.py tokeniza el archivo de entrada con tokenize() y ejecuta el ciclo shift/reduce/accept.

3.3 Algoritmo SLR: implementación detallada

Aumentación. Se añade la producción $S' \rightarrow S$ como producción 0. El símbolo aumentado S' permite que el estado de aceptación tenga un único ítem kernel.

Conjuntos FIRST y FOLLOW. Se calculan iterativamente hasta punto fijo. computeFirst soporta producciones vacías (épsilon); computeFollow añade \$ a FOLLOW(S') y propaga siguiendo las reglas estándar.

Clausura y goto. closure expande un conjunto de ítems LR(0) añadiendo todas las producciones de los no-terminales que aparecen después del punto. goto mueve el punto sobre un símbolo y

aplica clausura al resultado.

Construcción de estados. buildLr0States parte del estado $I0 = \text{closure}(\{S' \rightarrow \cdot S\})$ y aplica goto con todos los símbolos de la gramática, evitando duplicados mediante un mapa de conjuntos de ítems a identificadores de estado.

Tablas ACTION y GOTO. Shift cuando el punto está antes de un terminal; reduce sobre FOLLOW(A) cuando el punto está al final de $A \rightarrow \text{alfa}$; accept en el estado que completa $S' \rightarrow S$. Un conflicto ocurre cuando se intenta escribir dos acciones distintas en la misma celda; en ese caso SLRConflictError detiene el proceso.

3.4 Parser generado: theParser.py

codeGenerator.py serializa las tablas como literales Python y las inserta en una plantilla de texto. El archivo resultante contiene TOKENS, IGNORE_TOKENS, PRODUCTIONS, ACTION y GOTO como estructuras Python nativas, define _splitWithColumns() para tokenizar sin expresiones regulares (recorre carácter a carácter registrando columnas), implementa parse(tokens) con una pila de estados enteros, y puede ejecutarse directamente con `python theParser.py input.txt`.

3.5 Documentación del código

- **Nombres:** camelCase para funciones y variables; PascalCase para clases y dataclasses.
- **Dataclasses:** Production e Item son frozen=True para usarlos en conjuntos y como claves de diccionario.
- **Sin efectos secundarios:** las funciones del pipeline SLR son puras; toda la mutabilidad queda dentro de guiApp.py.
- **Sin expresiones regulares:** ningún módulo importa re. El reconocimiento de patrones se implementa con operaciones de strings (find, split, startswith) y recorridos de caracteres.

4. Cómo ejecutarlo

4.1 CLI

```
# Generar el parser
python yapar.py examples/medium/parser.yalp \
    -l examples/medium/lexer.yal -o theParser.py

# Ejecutar el parser generado
python theParser.py examples/medium/input_valid.txt
python theParser.py examples/medium/input_error.txt
```

4.2 Interfaz gráfica

```
python src/guiApp.py
```

La GUI ofrece las mismas capacidades que el CLI desde una sola ventana: carga de archivos (.yalp y .yal), validación YAPar/YALex, análisis de gramática, visualización del autómata LR(0), generación de theParser.py y prueba del parser con un archivo de entrada.

5. Archivos de prueba

Se proveen tres grupos de archivos, uno por nivel de complejidad. Cada grupo contiene: especificación de lexer (.yal), especificación de parser (.yalp), entrada válida e entrada con error.

Complejidad	Directorio	Gramática
Baja	examples/low/	expr -> NUM expr PLUS NUM
Media	examples/medium/	Aritmética con +, x, paréntesis
Alta	examples/high/	Asignaciones + IF...THEN...END